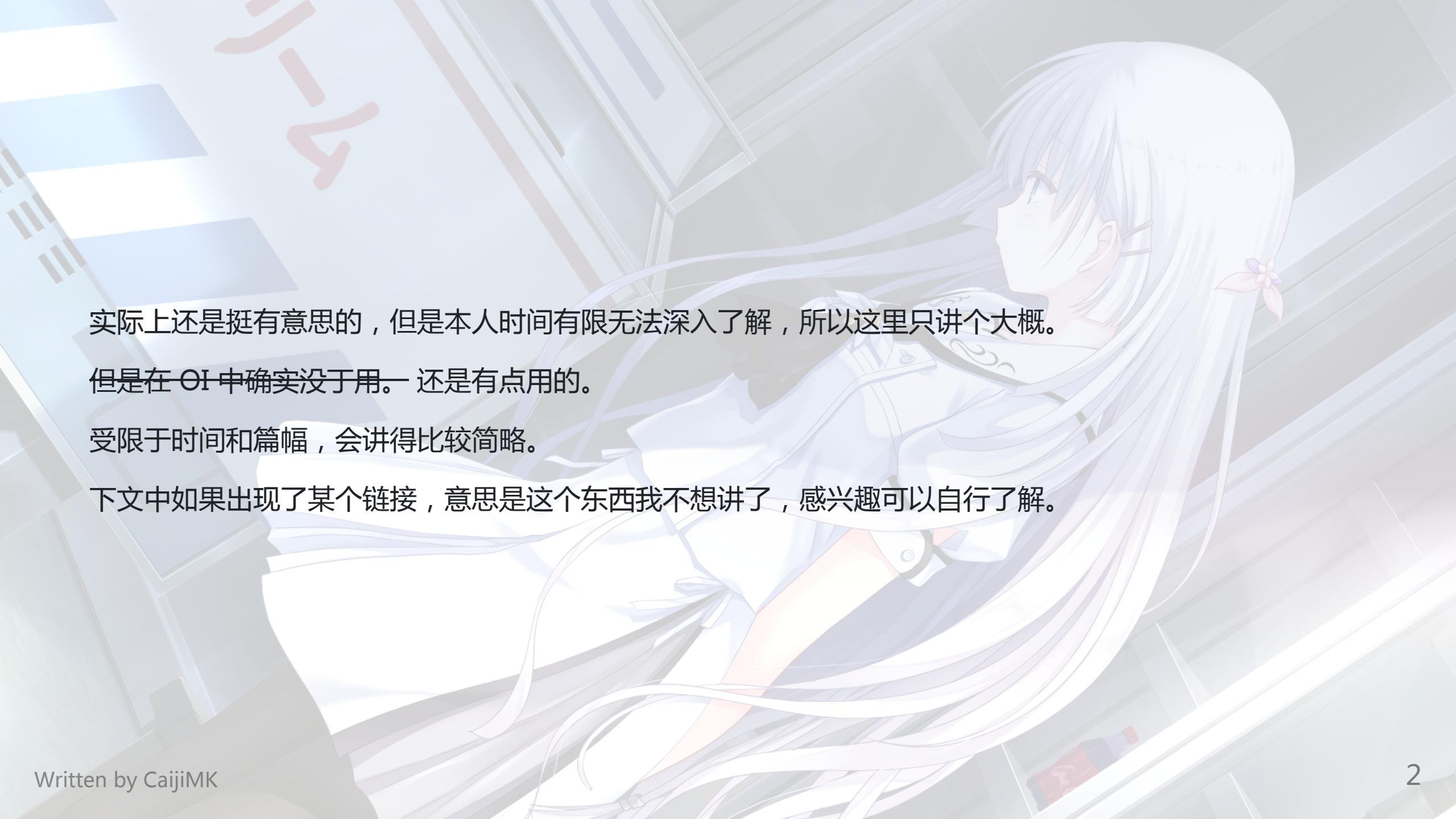


The background features a detailed illustration of a young woman with extremely long, flowing white hair. She is wearing a light blue, high-collared uniform with a white bow at the neck and a small pink flower accessory in her hair. She is looking upwards and to the right with a gentle expression. The setting appears to be a futuristic or industrial interior, with metallic structures and a large, stylized red Japanese character 'ハ' (Ha) visible in the upper left. The overall color palette is soft, dominated by blues, whites, and greys.

计算理论基础

2023.12.30



实际上还是挺有意思的，但是本人时间有限无法深入了解，所以这里只讲个大概。

但是在OI中确实没于用。 还是有点用的。

受限于时间和篇幅，会讲得比较简略。

下文中如果出现了某个链接，意思是这个东西我不想讲了，感兴趣可以自行了解。

形式语言

定义一个**字母表** (alphabet) 是一个非空有限集合, 用 Σ 表示, 其中元素被称为**符号 / 字符** (symbol)。

字符串是字母表中的元素构成的有穷序列。特别地, 用 ε 表示空串, 它是一个长度为 0 的不包含任何字符的串。

我们定义两个字符串 a, b 的**连接**为 ab 表示将 a 和 b 按顺序拼接在一起。

记所有长度为 n 的字符串的集合为 Σ^n , 特别地, $\Sigma^0 = \{\varepsilon\}$, 则还可以定义 $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$ 。它包含了所有 Σ 能构成的字符串。**语言** (language) 是 Σ^* 的一个子集。一般用 L 表示。

任何语言都可以转化成 01 串的形式。

语言的运算

我们可以定义语言上的运算：

- 交集 $\cap$, 并集 $\cup$, 补集。
- 串接： $AB = \{ab \mid a \in A, b \in B\}$, 有时也写作 $A \cdot B$ 。
- 幂运算： $L^0 = \{\varepsilon\}, \forall n \in \mathbb{N}^*, L^n = L^{n-1}L$ 。
- 闭包 (Kleene 星号)： $L^* = \bigcup_{i=0}^{\infty} L^i$ 。
- 右商运算： $A/B = \{x \mid \exists y \in B, xy \in A\}$, 类似的可以定义左商运算。
- 补语言：设 $L \in \Sigma^*$ 是一个语言 , 则 L 的补语言是 $\{x \mid x \in \Sigma^*, x \notin L\}$ 。

形式文法

不同于自然语言，我们需要一种明确的标准来定义一个字符串是否是某种形式语言的元素。对于大小有限的语言，我们显然可以通过枚举法来定义它。而对于大小无限的语言，我们可以通过**自动机**和**形式文法**来描述。

形式文法被定义为一个四元组 $G = (N, \Sigma, P, S)$ ：

- 有限的**非终结字符集** N ，与 G 生成的字符串无交。
- 有限的**终结字符集** Σ ，与 N 无交。
- 有限的**产生式规则集** P ，其每个元素都为如下形式：

$$(\Sigma \cup N)^* N (\Sigma \cup N)^* \rightarrow (\Sigma \cup N)^*$$

- **开始符号** S 。

我们定义形式文法 $G = (N, \Sigma, P, S)$ 内的字符串的二元关系 $\Rightarrow_G$:

$$x \Rightarrow_G y \iff \exists u, v, p, q \in (\Sigma \cup N)^*, x = upv, p \rightarrow q \in P, y = uqv$$

再定义二元关系 $\xRightarrow{*}_G$ 为 $\Rightarrow_G$ 的自反传递闭包。

通过这两个二元关系，我们可以定义**句型**为 $\{x \in (\Sigma \cup N)^* \mid S \xRightarrow{*}_G x\}$ 的元素。特别地，若 $x \in \Sigma^*$ ，则称 x 为**句子**。

一个形式文法生成的语言为句子的集合，记为 $L(G)$ ，即 $L(G) = \{x \in \Sigma^* \mid S \xRightarrow{*}_G x\}$ 。

乔姆斯基谱系

诺姆·乔姆斯基将形式文法分为四种类型：

- 0-型文法：可以识别递归可枚举语言，可以用图灵机识别。
- 1-型文法：可以识别上下文相关语言，可以用线性有界非确定图灵机识别。
- 2-型文法：可以识别上下文无关语言，可以用非确定下推自动机识别。
- 3-型文法：可以识别正则语言，可以用有限自动机识别。

其中 2-型 和 3-型 最为常用，因为它们的解析器可以方便且高效地实现，被称为上下文无关文法和正则语言文法。

上下文无关文法

要求 P 的元素必须是 $N \rightarrow (\Sigma \cup N)^*$ 的形式。可以用 [Earley 算法](#) 在 $O(n^3)$ 的时间复杂度内识别上下文无关语言。

正则文法

要求 P 的元素必须是 $N \rightarrow \Sigma$ 或 $N \rightarrow \Sigma N$ 的形式。

正则语言可以用有限状态自动机在线性时间内识别出来。

正则语言

正则语言有如下等价定义：

1. 可被确定有限状态自动机（DFA）识别。
2. 可被不确定有限状态自动机（NFA）识别。
3. 可被只读图灵机识别。
4. 可用正则表达式描述。
5. 可用正则文法生成。
6. 可用前缀文法生成。

正则表达式

正则表达式由常量和算子组成。对于字母表 Σ ，有如下常量：

- 空集 $\emptyset$ 。
- 空串 ε 。
- 文字字符 $a \in \Sigma$ ，表示 $\{a\}$ 。

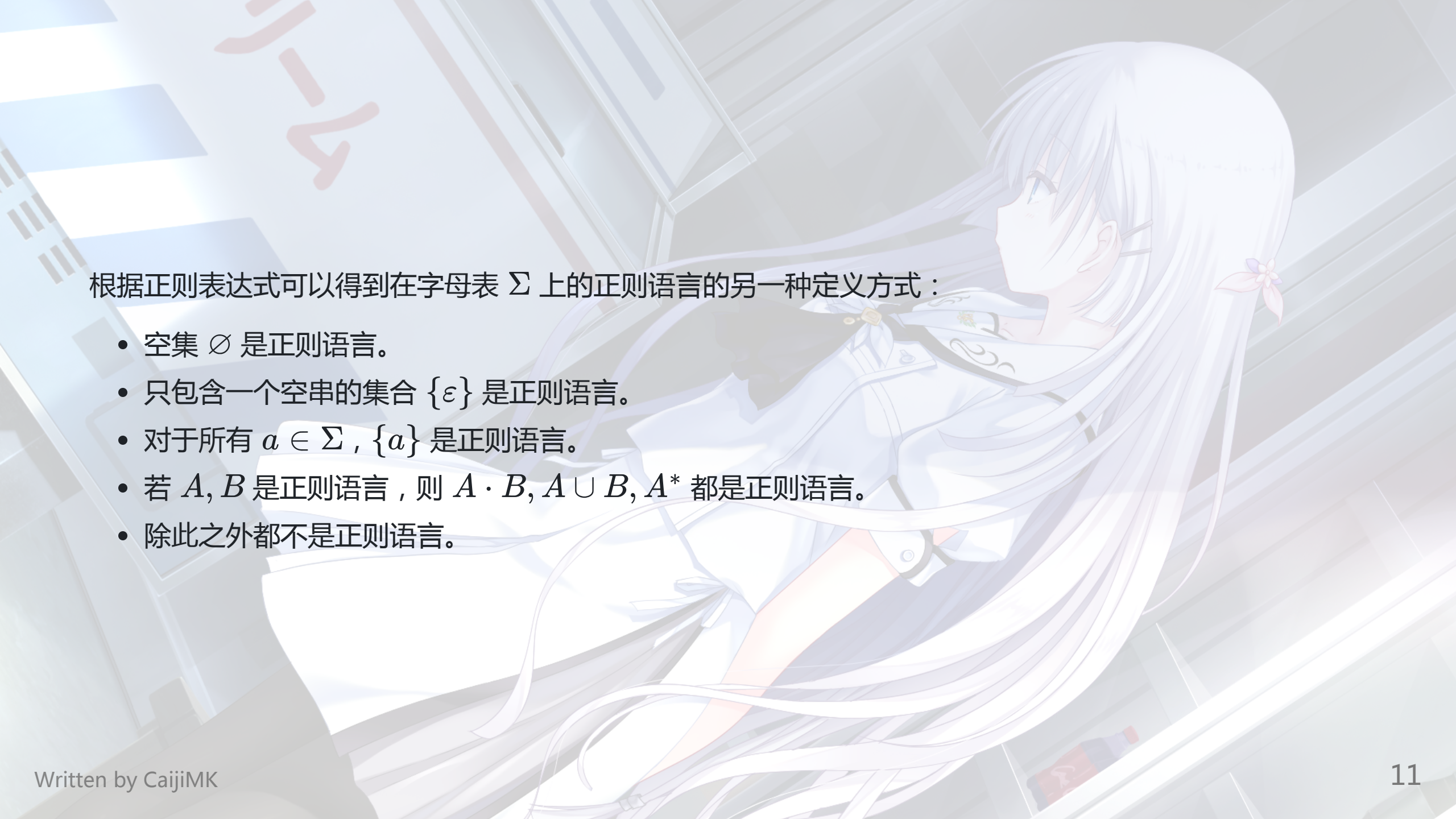
定义下列运算：

- 串接 AB ，等同于语言 A, B 的串接。
- 选择 $A|B = A \cup B$ ，有时也用 $\cup$ 或 $+$ 或 $\vee$ 代替。
- Kleene 星号 A^* 。

我们规定 Kleene 星号有最高优先级，其次是串接，选择最低。则可以通过上述常量和算子描述一个语言。

显然一个语言是正则语言当且仅当它可以用正则表达式描述。

Tips: 上述常量与算子构成了**克莱尼代数**。



根据正则表达式可以得到在字母表 Σ 上的正则语言的另一种定义方式：

- 空集 $\emptyset$ 是正则语言。
- 只包含一个空串的集合 $\{\epsilon\}$ 是正则语言。
- 对于所有 $a \in \Sigma$, $\{a\}$ 是正则语言。
- 若 A, B 是正则语言 , 则 $A \cdot B, A \cup B, A^*$ 都是正则语言。
- 除此之外都不是正则语言。

正则语言的封闭性

不(lan)加(de)证明地给出：

- 两个正则语言的串接、并集，正则语言的 Kleene 闭包是正则的。
- 两个正则语言的补、交和差是正则的。
- 两个正则语言的商是正则的。
- 正则语言的同态与逆同态是正则的。
-

有限状态自动机

有限状态自动机可以用来识别正则语言。其中 DFA 相关内容在另一个课件里已经出现过了，这里不再展开。

补充内容：在绘制自动机时，一般用一个没有起点的箭头指向起始状态，用同心圆表示终止状态。

NFA

对于两个自动机 A, B ，我们认为它们等价当且仅当 $\forall s, A(s) = B(s)$ 。

不确定有限状态自动机 (Non-deterministic Finite Automaton, NFA)，其与 DFA 不同之处在于 NFA 的一个转移可以对应多个状态，也就是说，其转移函数 δ 是一个 $(Q, \Sigma) \rightarrow 2^Q$ 的映射，其中 2^Q 表示 Q 的幂集。

其他定义与 DFA 相同。

NFA- ϵ (带 ϵ 移动的 NFA) 是一种扩展的 NFA，它允许使用空的符号进行转移，其转移函数为 $\delta : (Q, \Sigma \cup \{\epsilon\}) \rightarrow 2^Q$ 。

也就是在某个状态 u 可以转移到 $\delta(u, \epsilon)$ 中的某个状态而不消耗任何符号。

性质 1：普通 NFA 与 NFA- ϵ 是等价的，即任意一个 NFA- ϵ 也都可以找到一个与之等价的普通 NFA。

对于 NFA- ϵ 上的每个状态，求出其只通过 ϵ 转移能到的状态，然后将每个转移变成这些状态的转移的并即可。

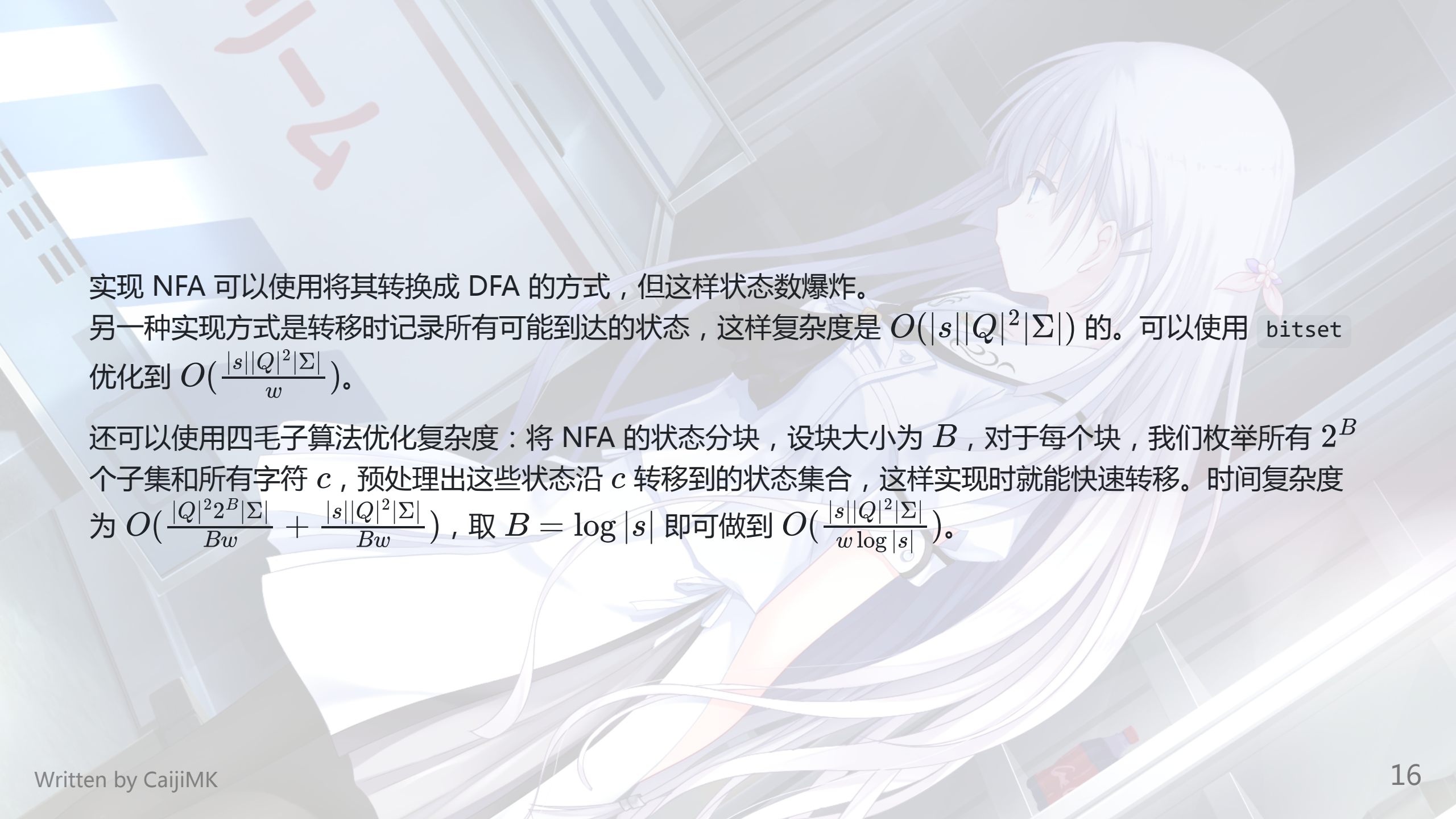
性质 2：NFA 与 DFA 是等价的。

将 NFA 转化成 DFA 的算法被称为子集构造法。

算法实现流程如下：

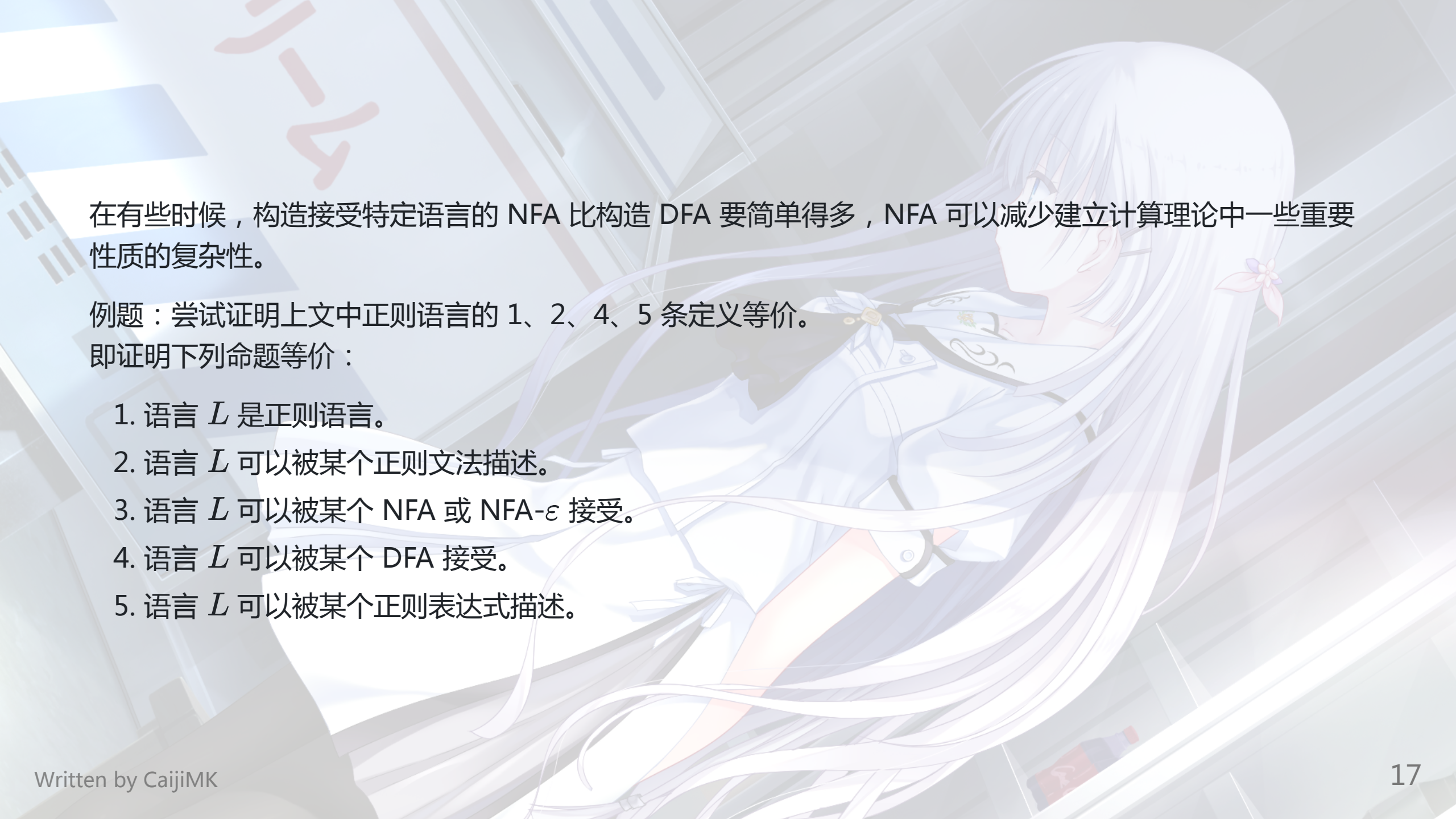
1. 将初始状态加入到 DFA 中但不标记。
2. 每次取出 DFA 中一个未被标记的状态，加上标记。
3. 将其转移设为其对应的 NFA 中的状态集合的转移的并对应的状态。
4. 若某个转移到的状态未被加入则加入 DFA 但不标记。
5. 重复 2 到 4 步直到所有状态都被标记。

正确性显然。但是这样构造出来的 DFA 的状态数是指数级别的。



实现 NFA 可以使用将其转换成 DFA 的方式，但这样状态数爆炸。
另一种实现方式是转移时记录所有可能到达的状态，这样复杂度是 $O(|s||Q|^2|\Sigma|)$ 的。可以使用 `bitset` 优化到 $O(\frac{|s||Q|^2|\Sigma|}{w})$ 。

还可以使用四毛子算法优化复杂度：将 NFA 的状态分块，设块大小为 B ，对于每个块，我们枚举所有 2^B 个子集和所有字符 c ，预处理出这些状态沿 c 转移到的状态集合，这样实现时就能快速转移。时间复杂度为 $O(\frac{|Q|^2 2^B |\Sigma|}{Bw} + \frac{|s||Q|^2|\Sigma|}{Bw})$ ，取 $B = \log |s|$ 即可做到 $O(\frac{|s||Q|^2|\Sigma|}{w \log |s|})$ 。



在有些时候，构造接受特定语言的 NFA 比构造 DFA 要简单得多，NFA 可以减少建立计算理论中一些重要性质的复杂性。

例题：尝试证明上文中正则语言的 1、2、4、5 条定义等价。
即证明下列命题等价：

1. 语言 L 是正则语言。
2. 语言 L 可以被某个正则文法描述。
3. 语言 L 可以被某个 NFA 或 NFA- ϵ 接受。
4. 语言 L 可以被某个 DFA 接受。
5. 语言 L 可以被某个正则表达式描述。

(1) $\iff$ (2) : 即定义。

(2) $\implies$ (3) : 考虑构造一个带 ε 移动的 NFA : 起始状态为 S , 对于每一条 $A \rightarrow cB, A, B \in N, c \in \Sigma$ 的规则 , 有转移 $\delta(A, c) = B$, 对于每一条 $A \rightarrow c, A \in N, c \in \Sigma$, 有转移 $\delta(A, c) = F$, 其中 F 是一个终结状态。

(3) $\implies$ (2) : 类似地 , 将每个状态都对应一个非终结字符 , 起始状态对应 S , $\forall A, B \in Q, c \in \Sigma, B \in \delta(A, c)$, 有规则 $A \rightarrow cB \in P$, 对于任意最终状态 $f \in F$, 有规则 $f \rightarrow \varepsilon \in P$ 。

(3) $\iff$ (4) : 上文已经证明。

(5) $\implies$ (3) : 按照运算顺序归纳证明。

对于常量 :

- 空集 : 建立自动机 $Q = \{A, B\}, S = A, F = \{B\}$, A, B 之间没有转移。
- ε : 建立自动机 $Q = \{A, B\}, S = A, F = \{B\}$, 转移为 $\delta(A, \varepsilon) = B$ 。
- 单个字符 a : 建立自动机 $Q = \{A, B\}, S = A, F = \{B\}$, 转移为 $\delta(A, a) = B$ 。

对于算子 : (以下 A 表示前一个表达式对应的自动机 , B 表示后一个表达式对应的自动机 , 用 $S(A)$ 表示自动机 A 的起始状态 , $Q(A), F(A)$ 同理)

- 串接 : $\forall f \in F(A)$, 增加一个转移 $\delta(f, \varepsilon) = S(B)$, 并且对于新的自动机 , $S = S(A), F = F(B)$ 。
- 并集 : 新建状态 S' , $\delta(S', \varepsilon) = \{S(A), S(B)\}$, 并且对于新的自动机 , $S = S', F = F(A) \cup F(B)$ 。
- Kleene 星号 : 对于自动机 A , $\forall f \in F(A)$, 增加一个转移 $\delta(f, \varepsilon) = S(A)$, 并增加转移 $\delta(S(A), \varepsilon) = S(A)$ 。

(3) $\implies$ (5): (这个神中神！)

给状态编号，用 $R_{i,j,k}$ 表示从 i 到 j ，除起点和终点外只经过编号 $\leq k$ 的状态构成的语言的正则表达式。

- $\forall k > 0, R_{i,j,k} = R_{i,j,k-1} | R_{i,k,k-1} R_{k,k,k-1}^* R_{k,j,k-1}$ 。
- $R_{i,j,0} = \bigcup_{a,j \in \delta(i,a)} a$ 。

则该自动机的正则表达式即为 $\bigcup_{f \in F} R_{S,f,|Q|}$ 。

□

DFA 最小化

将一个 DFA 转化为与它等价且状态数最少的 DFA，使用 Hopcroft 算法。

首先去除所有无法从 s 到达的和无法到达 F 的状态。

我们定义两个状态 u, v 是不可区分的，当且仅当：

- $[u \in F] = [v \in F]$ ，且
- $\forall c \in \Sigma, \delta(u, c)$ 与 $\delta(v, c)$ 相同或不可区分。

具体实现时，我们先将状态按是否是最终状态划分为两个集合，每次取出一个集合 S 使得存在状态 $u, v \in Q$ 和字符 $c \in \Sigma$ ，满足在当前划分情况下 $\delta(u, c)$ 与 $\delta(v, c)$ 不在同一集合中。则按照转移函数继续划分 S ，直到找不到满足条件的 S 。

可以证明不存在两个不可区分的状态的 DFA 是最小的。

这里应该有个证明，如果忘记补了就对着[这个](#)讲。

DFA & NFA 在 OI 中的应用

应用一：构建 DFA 以识别某些特定的语言

CF1142D Foreigner

定义一个数 x 是牛的当且仅当：

- x 是 1, 2, 3, 4, 5, 6, 7, 8, 9, 或
- $\lfloor \frac{x}{10} \rfloor$ 是牛的，并且将所有牛的数从小到大排序后， $\lfloor \frac{x}{10} \rfloor$ 的排名 k (排名从 1 开始) 满足 $x \bmod 10 < k \bmod 11$ 。

给你一个由数码构成的字符串 s ，求有多少子串不含前导零且对应的数是牛的。

$$1 \leq |s| \leq 10^5.$$

观察条件，可以发现除 $1 \sim 9$ 以外的牛的数都能由其他牛的数在末尾添加一个数码得到。

具体地，若一个牛的数 x 的排名为 k ，则它能生成 $10x \sim 10x + k \bmod 11 - 1$ 的数。设 $x' = 10x + c (0 \leq c < k \bmod 11)$ ， x' 的排名 k' 可以方便地用以下式子求出：

$$k' = 9 + \sum_{i=1}^{k-1} (i \bmod 11) + c + 1$$

观察发现：

$$(0 \bmod 11) + (1 \bmod 11) + \dots + (10 \bmod 11) = 55 \equiv 0 \pmod{11}$$

所以我们只需要知道 $k \bmod 11$ 的值即可。

基于这点，我们建立一个只接受牛的数的 DFA：

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$Q = \{s, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$F = Q \setminus \{s\}$$

$$\delta(u, c) = \begin{cases} c & u = s, c \neq 0 \\ (10 + \sum_{i=0}^{u-1} i + c) \bmod 11 & u \neq s, c < u \end{cases}$$

设 $dp_{i,u}$ 表示只考虑以 $\text{pre}(s, i)$ 为后缀的串，在 DFA 上的状态为 u 时的方案数，非常容易写出转移方程。

最终答案即为 $\sum_{i=1}^{|s|} \sum_{u=0}^{10} dp_{i,u}$ 。

应用二：DP 套 DP

一般用来解决需要满足某种 ald 限制的计数。

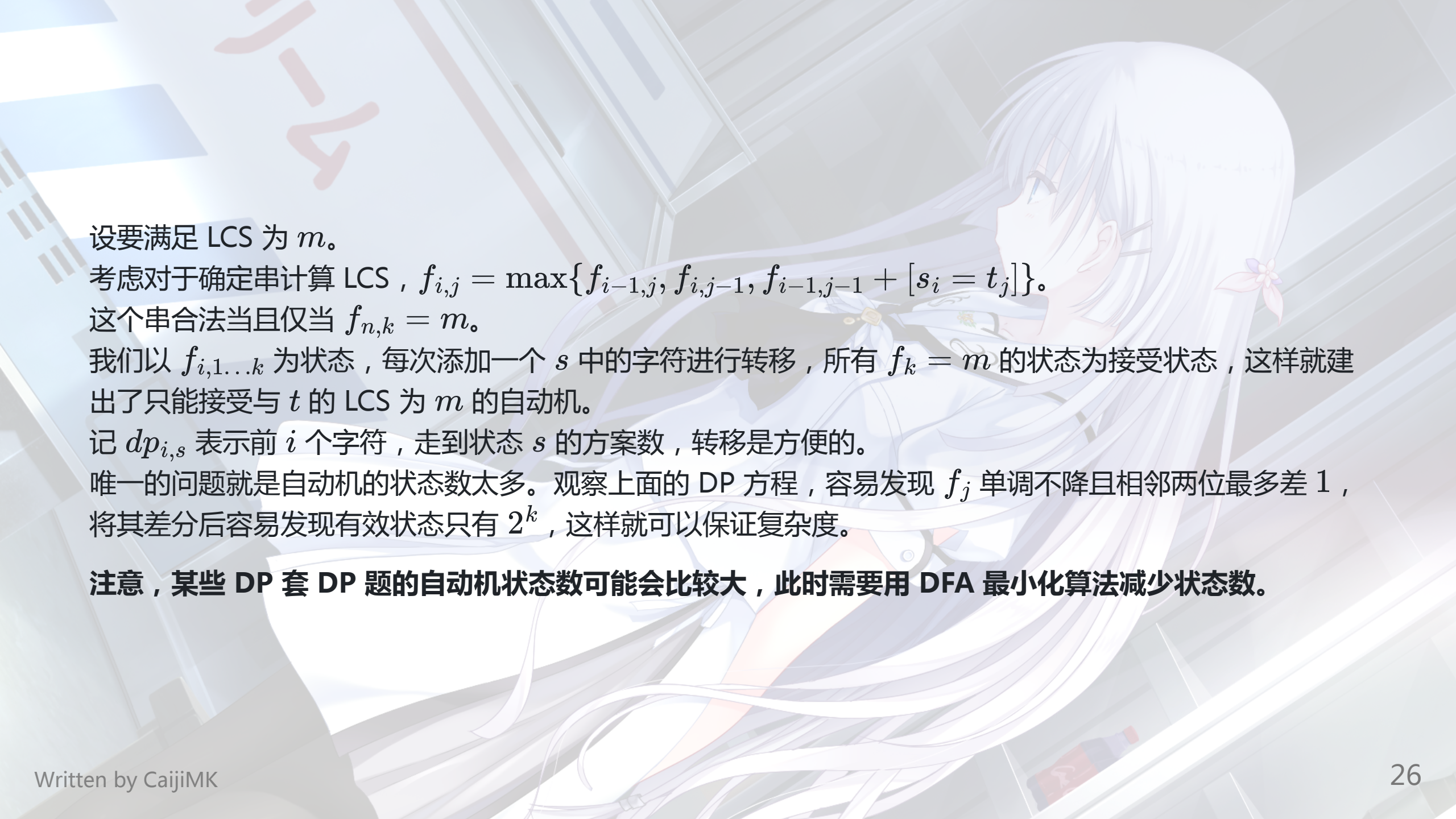
大致思路是将限制用自动机描述，然后将自动机的状态存到 DP 的状态中以解决限制问题。一般内部自动机的构建也需要 DP 来完成。

「TJOI2018」游园会

给你一个长度为 k ，字符集为 $\{N, O, I\}$ 的串 t ，求长度为 n ，字符集为 $\{N, O, I\}$ 的串 s 的数量，满足 s 与 t 的 LCS 为 i ，且不存在一个子串为 NOI。

对于所有的 i 求解。

$$1 \leq n \leq 1000, 1 \leq k \leq 15.$$



设要满足 LCS 为 m 。

考虑对于确定串计算 LCS, $f_{i,j} = \max\{f_{i-1,j}, f_{i,j-1}, f_{i-1,j-1} + [s_i = t_j]\}$ 。

这个串合法当且仅当 $f_{n,k} = m$ 。

我们以 $f_{i,1\dots k}$ 为状态, 每次添加一个 s 中的字符进行转移, 所有 $f_k = m$ 的状态为接受状态, 这样就建出了只能接受与 t 的 LCS 为 m 的自动机。

记 $dp_{i,s}$ 表示前 i 个字符, 走到状态 s 的方案数, 转移是方便的。

唯一的问题就是自动机的状态数太多。观察上面的 DP 方程, 容易发现 f_j 单调不降且相邻两位最多差 1, 将其差分后容易发现有效状态只有 2^k , 这样就可以保证复杂度。

注意, 某些 DP 套 DP 题的自动机状态数可能会比较大, 此时需要用 DFA 最小化算法减少状态数。

正则语言的泵引理

「在可计算性理论中的形式语言理论中，泵引理（Pumping lemma）声称给定类的任何语言可以被“抽吸”并仍属于这个类。一个语言可以被抽吸，如果在这个语言中任何足够长的字符串可以分解成片段，其中某些可以任意重复来生成语言中更长的字符串。这些引理的证明典型的需要计数论证比如鸽笼原理。」

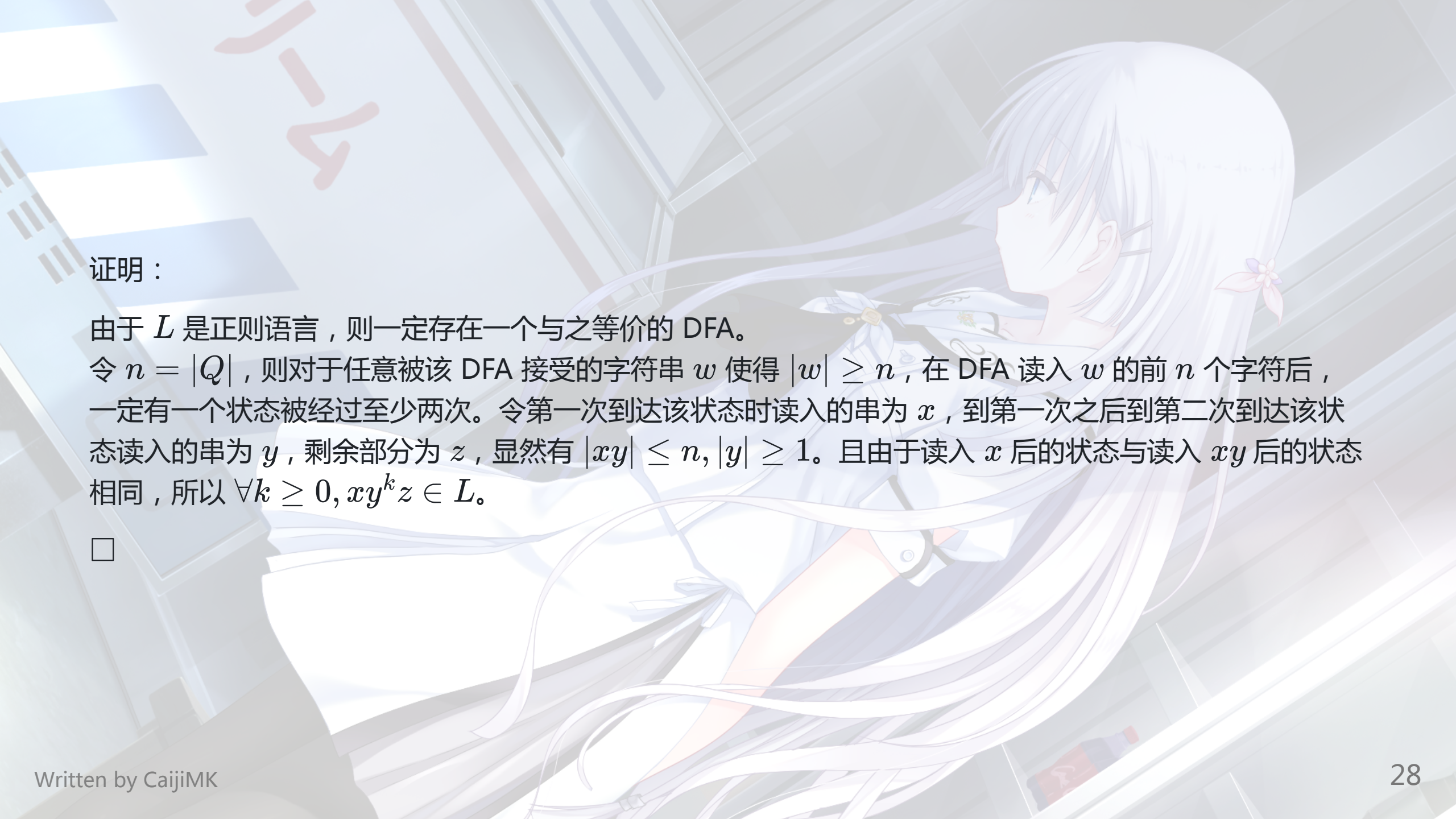
—— Wikipedia

依托批话，看也看不懂在说什么。

我们只要看正则语言的泵引理就好了。

Pumping Lemma：假设 $L \subseteq \Sigma^*$ 是一个正则语言，则存在正整数 $n \geq 1$ （称为泵长度），则 $\forall w \in L, |w| \geq n, \exists x, y, z, w = xyz$ ，使得以下说法成立：

- $|xy| \leq n$
- $|y| \geq 1$
- $\forall k \geq 0, xy^kz \in L$

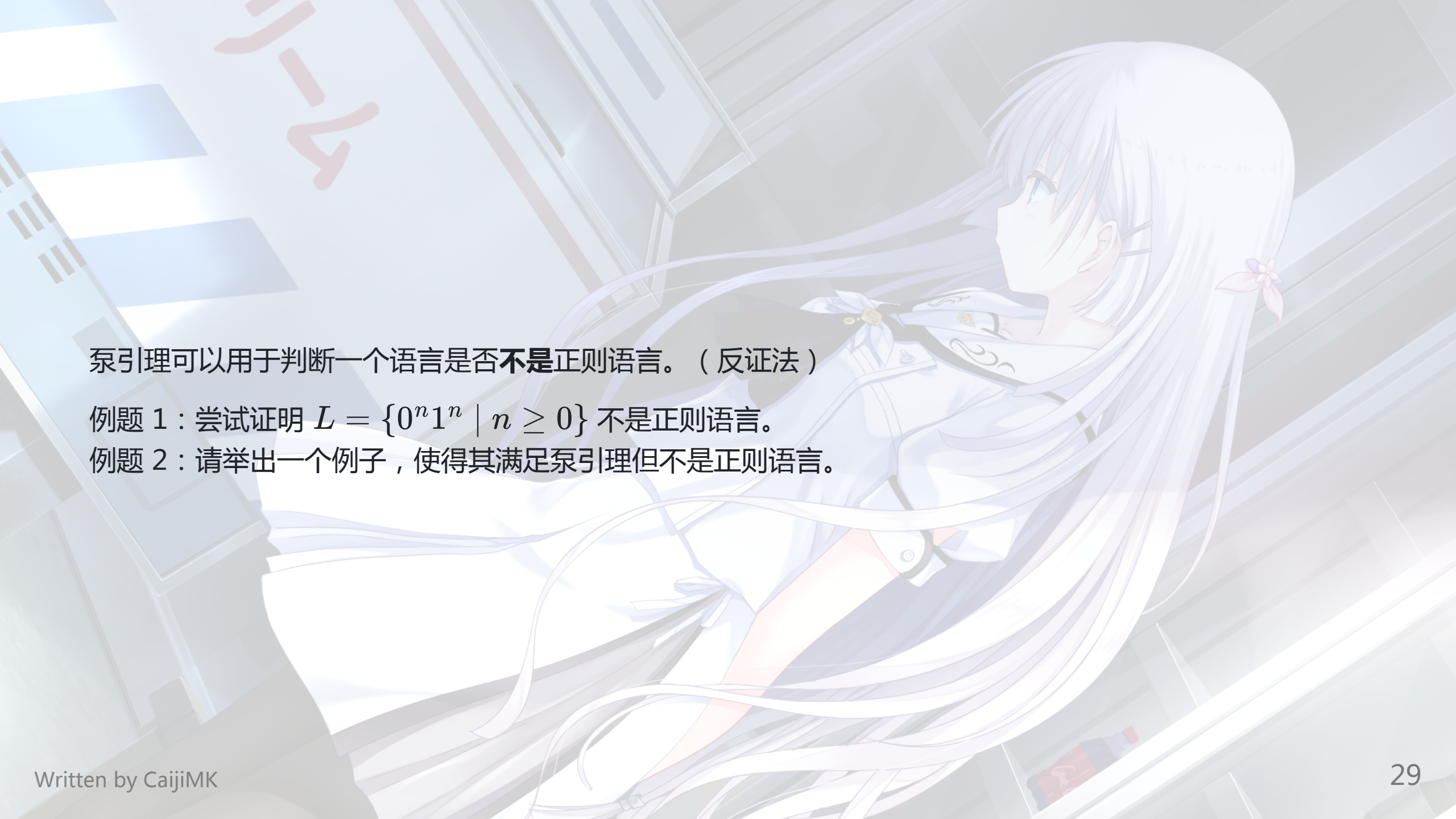


证明：

由于 L 是正则语言，则一定存在一个与之等价的 DFA。

令 $n = |Q|$ ，则对于任意被该 DFA 接受的字符串 w 使得 $|w| \geq n$ ，在 DFA 读入 w 的前 n 个字符后，一定有一个状态被经过至少两次。令第一次到达该状态时读入的串为 x ，到第一次之后到第二次到达该状态读入的串为 y ，剩余部分为 z ，显然有 $|xy| \leq n, |y| \geq 1$ 。且由于读入 x 后的状态与读入 xy 后的状态相同，所以 $\forall k \geq 0, xy^kz \in L$ 。

□



泵引理可以用于判断一个语言是否**不是**正则语言。（反证法）

例题 1：尝试证明 $L = \{0^n 1^n \mid n \geq 0\}$ 不是正则语言。

例题 2：请举出一个例子，使得其满足泵引理但不是正则语言。



例题 1：设泵长度为 n ，取 $w = a^n b^n$ 即可。

例题 2：一个例子是 $L = \{a^m b^n c^n \mid m, n \geq 0\}$ 。

图灵机和复杂度类

图灵机一般指确定型图灵机，可以证明其他变种与确定型图灵机的计算能力是一样的。

确定型图灵机

一台图灵机可以被描述为一个七元组 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ ：

- 非空有穷的状态集合 Q 。
- 非空有穷的输入字母表 Σ ，其不包含空字符 $\square$ 。
- 非空有穷带字母表 Γ ，满足 $\Sigma \subset \Gamma, \square \in \Gamma \setminus \Sigma$ 。
- 转移函数 $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ 。
- 起始状态 $q_0 \in Q$ 。
- 接收状态 $q_{accept} \in Q$ 。
- 拒绝状态 $q_{reject} \in Q$ 。

图灵机的工作流程：

图灵机有一个长度无限的纸带 TAPE，一个读写头 HEAD 指向纸带上某个格子和一个状态寄存器维护当前所在的状态。

初始时将输入串 $s = s_0s_1 \dots s_{|s|-1} \in \Sigma^*$ 从左往右依次填在纸带的第 $0, 1, \dots, |s| - 1$ 个格子上，其他格子保持空白。读写头指向第 0 个格子，图灵机处在状态 q_0 。

设当前状态为 q ，读写头指向的格子上的字符为 x ，设 $\delta(q, x) = (q', x', D)$ ，则图灵机会将状态移动到 q' ，将读写头指向的格子上的字符改为 x' 。若 $D = L$ ，则将读写头向左移动一格，否则向右移动一格。特别地，若读写头处在 0 号格子且 $D = L$ ，则不会进行移动。

若某个时刻进入接受状态，则立刻停机并接受该串。若某个时刻可进入拒绝状态，则立刻停机并拒绝该串。

注意：某些 $\delta(q, x)$ 是未定义的，此时图灵机立刻停机。

图灵机 M 在输入为 x 时的输出记为 $M(x)$ ， $M(x) = 1$ 当且仅当 M 接受 x ， $M(x) = 0$ 当且仅当 M 拒绝 x 。

里面可以有多个参数，实现时用一个字符表示「逗号」即可。

图灵机可以被自然数标号，因此图灵机的数量是可数无穷的。

以下记编号为 α 的图灵机为 M_α 。

丘奇-图灵论题

图灵认为图灵机可以模拟人类所能进行的任何计算过程。

更具体地，若一类问题有一个有效的方法解决，则这类问题可以被某个图灵机解决。

其中，「有效的方法」需要满足：

- 包含有限条清晰的指令；
- 当用其解决这类问题的其中一个时，这个方法需要在有限步骤内结束，且得到正确的答案。

这个论题被称为**丘奇-图灵论题**，这个论题**没有**被证明，但它是计算理论的一条基本公理。



通用图灵机

因为图灵机可以用自然数编号，所以可以设计出一台图灵机先接受一个编码 α ，然后模拟 M_α 进行工作，我们把这台图灵机称为通用图灵机。

判定问题

判定问题是只能回答 YES / NO 的问题，本质上是判断某个串是否属于某个语言。

一个语言可以代指「判定一个串是否属于这个语言」这个判定问题，因此语言和问题可以认为是同义词。

功能性问题

相较于判定问题，功能性问题的回答不止 YES / NO，可以是别的什么东西（比如一个数字）。

显然扔和一个判定问题都是功能性问题。也就是说，任何一个问题的判定都不难于求值。

可计算性

我们认为一个判定问题是**图灵可计算问题**当且仅当存在一个总是在有限步内停机且正确判断的图灵机。不存在这样一个图灵机的判定问题被称为**图灵不可计算问题**。

由于图灵机的数量是可数无穷的，而语言（无限长的二进制串）的数量是不可数无穷，所以一定存在图灵不可计算问题。

一个经典的例子是停机问题：给定 α 和 x ，判断 M_α 能否在有限步内停机。

停机问题是图灵不可计算问题的证明：

定义函数 $UC : \{0, 1\}^* \rightarrow \{0, 1\}$ 为：

$$UC(\alpha) = \begin{cases} 0 & M_\alpha(\alpha) = 1 \\ 1 & \text{otherwise} \end{cases}$$

假设存在图灵机 M_β 可以计算 UC ，则 $UC(\beta) = 1 \iff M_\beta(\beta) \neq 1$ ，但因为 M_β 能计算 UC ，所以 $M_\beta(\beta) = UC(\beta)$ ，因此不存在可以计算 UC 的图灵机。

令 M_{HALT} 是能计算停机问题的图灵机， $M_{\text{HALT}}(\alpha, x)$ 的值为 1 当且仅当 M_α 在输入 x 时可以停机，则可以计算一台能计算 UC 的图灵机 M_{UC} ：

M_{UC} 调用 $M_{\text{HALT}}(\alpha, \alpha)$ ，如果输出为 0，则 $M_{\text{UC}}(\alpha) = 1$ ，否则，调用通用图灵机计算答案。

因此， M_{HALT} 不存在，所以停机问题是图灵不可计算问题。

□

复杂度类

CNM, BUXIELE

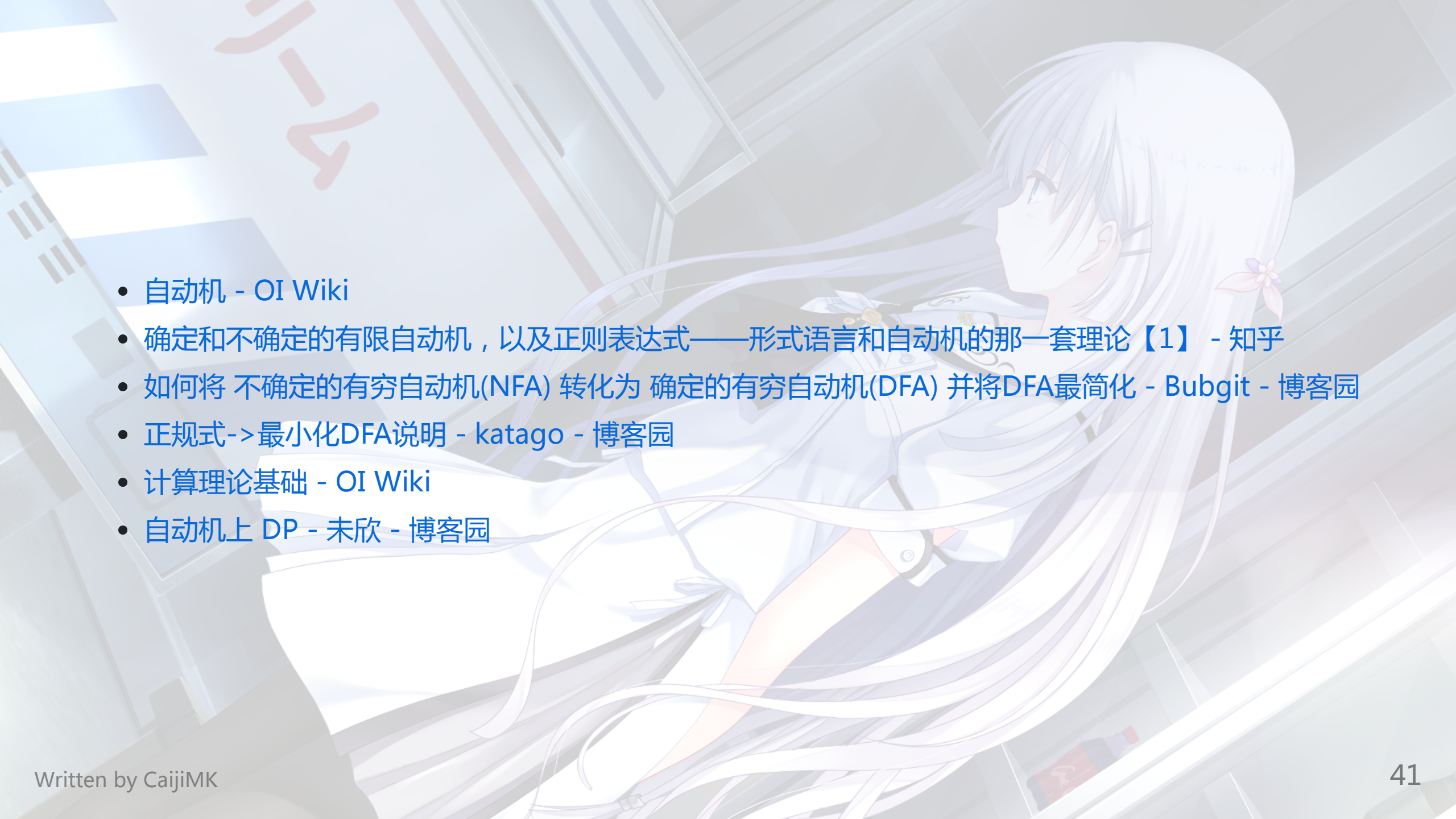
弃疗了，我们直接对着参考资料讲。

推荐题目

- [CF1142D Foreigner](#)
- 「TJOI2018」游园会
- [CF956F Minimal Subset Difference](#)

参考资料

- [形式语言 - 维基百科，自由的百科全书](#)
- [形式文法 - 维基百科，自由的百科全书](#)
- [乔姆斯基谱系 - 维基百科，自由的百科全书](#)
- [正则语言 - 维基百科，自由的百科全书](#)
- [正则表达式 - 维基百科，自由的百科全书](#)
- [非确定有限状态自动机 - 维基百科，自由的百科全书](#)
- [泵引理 - 维基百科，自由的百科全书](#)

- 
- [自动机 - OI Wiki](#)
 - [确定和不确定的有限自动机，以及正则表达式——形式语言和自动机的那一套理论【1】 - 知乎](#)
 - [如何将 不确定的有穷自动机\(NFA\) 转化为 确定的有穷自动机\(DFA\) 并将DFA最简化 - Bubgit - 博客园](#)
 - [正规式->最小化DFA说明 - katago - 博客园](#)
 - [计算理论基础 - OI Wiki](#)
 - [自动机上 DP - 未欣 - 博客园](#)



GL & HF